

PIC1 Final Report

While there were many open-source projects we were interested in and wanted to contribute to because we believed in their mission, finding a project suited for this class's assignment was a little more challenging than we initially thought. Most projects we had in mind were either not big or active enough, or they required technical knowledge that was difficult to study in just a couple of weeks. Eventually, however, we were able to find one that we liked and met the needed requirements, and we started working on it.

Finding the bugs was fairly easy since our project, mpv, had a category for bugs in the issues page of the GitHub repository, and there were many issues open that we could choose from. Fixing the bugs was also relatively straightforward as it only required small code changes and the users that reported the bugs described them well, providing clear reproduction steps as well as the resulting log files. However, creating tests to verify these fixes was much more challenging, since the existing test set is sparse and lacks clear documentation on how to properly implement new tests.

The feature we proposed was the implementation of an alternative OSD (On-Screen Display) written in Lua so it would be easier for third-party developers to make their own configuration files for the video player's GUI. This was precisely what we implemented, via a Lua script, something mpv already easily integrates into its own codebase. Our feature required a couple of different steps and different aspects of mpv's code that we had to change, such as adding a new command line option to run mpv with, suppressing the OSD in C, loading and activating the new Lua script inside mpv and writing the Lua OSD itself.

Our predictions in terms of work effort weren't too far off. The project took a few hours longer than expected during implementation, mainly because we initially felt somewhat disoriented and had to familiarize ourselves with the 'player' folder and the hooks it provides to interact with mpv. We also needed time to learn how to use Lua and better understand what was expected from our OSD implementation. However, debugging was fortunately simpler and matched our predicted efforts, as we received very fast visual feedback from our code changes simply by starting up a video. Implementing the tests went also as expected since we had already gained some experience with them from our earlier bug fixes. Hence, we estimate that we spent around 22 hours in implementation and 10 hours in testing and debugging,

On the other hand, we ended up writing more lines of code than anticipated, especially due to the OSD requiring many variable declarations, and because each time we created a new ASS object, we had to set their attributes according to our configured options. Though as a result this made every object even more customizable, which meant we were able to provide more extensive configuration options and improve the core purpose of our feature.

Our interactions with the open-source community during this assignment were not the most positive. We encountered some resistance to change and we felt like the developers preferred to leave known bugs in the code rather than fixing them, due to the available solutions not being ideal. What we found stranger, however, was that despite these being known bugs and the developers not wanting to fix them, the issues where they were reported were still left open. Naturally, every project's community is different, and even in this one it is up to each contributor to implement good enough solutions to be accepted. We merely expected the developers to be overall more receptive and welcoming, and were surprised with what we found, although this too is understandable since development is done by volunteers and it is important to direct efforts to more critical parts of the project.

With the development of our feature and bug fixes we definitely learned a lot. Going through such a large and interconnected codebase with many different components was difficult and it was definitely the biggest challenge we encountered. Having to go through this process, even though daunting at first, was good and helped us improve this much needed skill of analyzing other people's code. We also learned to program integration tests in C as well as to code in Lua, a language we had little experience with before, and will definitely prove to be useful in the future since it is a language commonly used while embedded in larger projects to do smaller tasks like scripting or modding.

Regarding the project's documentation, we are definitely of the opinion that it is not good enough for newcomers and it could be improved. Both the Git master and the Stable release's reference manuals (found here - <https://mpv.io/manual/>) are directed towards everyday users and people who seek to customize or configure mpv, providing a very extensive list of options and their meanings. However, the internal functions of mpv are not documented and the overall code lacks comments. If someone has a doubt about what a specific function does or how it works, there is no place they can go look for that information, and the only option is to continue to relentlessly analyze the code. This makes it harder for new contributors to get started, as mpv is a project with several moving pieces and dozens of files about an already complicated topic that is video playing. Thus, this is what we would improve in this project, despite these not being actual features or programming, because we believe that good documentation and an easy to understand codebase are vital for gathering more people to help with a project.

Overall, we enjoyed working on this project as it was quite different from the scope we were used to, both in terms of coding and in terms of interacting with a different environment, even if we ended up with a few minor complaints. We don't plan to contribute any other features for now, and will instead focus solely on refining this one after the feedback received from the developers when we submit a pull request.